

SSGMCE	SHRI SANT GAJANAN MAHARAJ COLLEGE OF ENGG.		LABORATORY MANUAL	
	PRACTICAL EXPERIMENT INSTRUCTION SHEET			
	EXPERIMENT TITLE : To study and Implement Knapsack problem using greedy method.			
EXPERIMENT NO. : SSGMCE/WI/IT/01/6IT07/05		ISSUE NO. : 00	ISSUE DATE : 01.07.2023	
REV. DATE :		REV. NO. :	DEPTT. : INFORMATION TECHNOLOGY	
LABORATORY : Design and Analysis of Algorithm			SEMESTER : VI	PAGE: 1 OF 4

1.0) AIM:

To study and Implement Knapsack problem using greedy method

2.0) SCOPE:**3.0) FACILITIES/ APPARATUS:**

i) **Software:** Turbo C

4.0) THEORY:

Greedy is an algorithmic paradigm that builds up a solution piece by piece, always choosing the next piece that offers the most obvious and immediate benefit. So the problems where choosing locally optimal also leads to global solution are the best fit for Greedy.

For example consider the [Fractional Knapsack Problem](#). The local optimal strategy is to choose the item that has maximum value vs weight ratio. This strategy also leads to a globally optimal solution because we are allowed to take fractions of an item.



Given the weights and profits of N items, in the form of {profit, weight} put these items in a knapsack of capacity W to get the maximum total profit in the knapsack. In Fractional Knapsack, we can break items for maximizing the total value of the knapsack.

Input: $arr[] = \{\{60, 10\}, \{100, 20\}, \{120, 30\}\}, W = 50$

Output: 240

Explanation: By taking items of weight 10 and 20 kg and 2/3 fraction of 30 kg.

Hence total price will be $60 + 100 + (2/3)(120) = 240$

Input: $arr[] = \{\{500, 30\}\}, W = 10$

Output: 166.667

Time Complexity: $O(2^N)$

Auxiliary Space: $O(N)$

Fractional Knapsack Problem using [Greedy algorithm](#):

An efficient solution is to use the Greedy approach.

The basic idea of the greedy approach is to calculate the ratio **profit/weight** for each item and sort the item on the basis of this ratio. Then take the item with the highest ratio and add them as much as we can (can be the whole element or a fraction of it).

This will always give the maximum profit because, in each step it adds an element such that this is the maximum possible profit for that much weight.

Illustration:

Check the below illustration for a better understanding:

Consider the example: $arr[] = \{\{100, 20\}, \{60, 10\}, \{120, 30\}\}, W = 50$.

Sorting: Initially sort the array based on the profit/weight ratio. The sorted array will be $\{\{60, 10\}, \{100, 20\}, \{120, 30\}\}$.

Iteration:

- For $i = 0$, weight = 10 which is less than W . So add this element in the knapsack. **profit** = 60 and remaining $W = 50 - 10 = 40$.
- For $i = 1$, weight = 20 which is less than W . So add this element too. **profit** = 60 + 100 = 160 and remaining $W = 40 - 20 = 20$.
- For $i = 2$, weight = 30 is greater than W . So add $20/30$ fraction = $2/3$ fraction of the element. Therefore **profit** = $2/3 * 120 + 160 = 80 + 160 = 240$ and remaining W becomes 0.

So the final profit becomes 240 for $W = 50$.

Follow the given steps to solve the problem using the above approach:

- Calculate the ratio (**profit/weight**) for each item.
- Sort all the items in decreasing order of the ratio.
- Initialize **res** = 0, **curr_cap** = given_cap.
- Do the following for every item i in the sorted order:
 - If the weight of the current item is less than or equal to the remaining capacity then add the value of that item into the result
 - Else add the current item as much as we can and break out of the loop.
- Return **res**.

Example:

A knapsack is a bag. And the knapsack problem deals with the putting items to the bag based on the value of the items. It aim is to maximise the value inside the bag. In 0-1 Knapsack you can either put the item or discard it, there is no concept of putting some part of item in the knapsack.

Sample Problem

Value of items = {20, 25, 40}

Weights of items = {25, 20, 30}

Capacity of the bag = 50

Weight distribution

25, 20 {1, 2}

20, 30 {2, 3}

If we use {1, 3} the weight will be above the max allowed value.

For {1, 2} : weight = 20 + 25 = 45 Value = 20 + 25 = 45

For {2, 3} : weight = 20 + 30 = 50 Value = 25 + 40 = 65

The maximum value is 65 so we will put the item 2 and 3 in the knapsack.

PROGRAM FOR FRACTION KNAPSACK PROBLEM

```
#include <stdio.h>
int n = 5;
int p[10] = {3, 3, 2, 5, 1};
int w[10] = {10, 15, 10, 12, 8};
int W = 10;
int main(){
    int cur_w;
    float tot_v;
    int i, maxi;
    int used[10];
    for (i = 0; i < n; ++i)
        used[i] = 0;
    cur_w = W;
    while (cur_w > 0) {
        maxi = -1;
        for (i = 0; i < n; ++i)
            if ((used[i] == 0) &&
                ((maxi == -1) || ((float)w[i]/p[i] >
                    (float)w[maxi]/p[maxi])))
                maxi = i;
        used[maxi] = 1;
        cur_w -= p[maxi];
        tot_v += w[maxi];
        if (cur_w >= 0)
            printf("Added object %d (%d, %d) completely in the bag.
Space left: %d.\n", maxi + 1, w[maxi], p[maxi], cur_w);
```

EXPERIMENT NO. : SSGMCE/WI/IT/01/6IT07/05	ISSUE NO. : 00	ISSUE DATE : 01.07.2023
LABORATORY : Design and Analysis of Algorithm Lab (6IT07)	SEMESTER : VI	PAGE: 4 OF 4

```
        else {
            printf("Added %d%% (%d, %d) of object %d in the
bag.\n", (int)((1 + (float)cur_w/p[maxi]) * 100), w[maxi],
p[maxi], maxi + 1);
            tot_v -= w[maxi];
            tot_v += (1 + (float)cur_w/p[maxi]) * w[maxi];
        }
    }
    printf("Filled the bag with objects worth %.2f.\n", tot_v);
    return 0;
}
```

5.0) Conclusion:

Implemented Knapsack problem using greedy method.

PREPARED BY:
PROF.A.G.SHARMA

APPROVED BY: (H.O.D.)
DR. A. S. MANEKAR